

Absolutely. At this point you have a good foundation for a **local Kubernetes + Terraform + Kind + GitHub + Argo CD GitOps lab**, so I would document it as a reusable lab runbook rather than just a list of commands.

Below is a professional reference document you can save as:

```
~/git/k8s-gitops-lab/docs/KUBERNETES-GITOPS-LAB.md
```

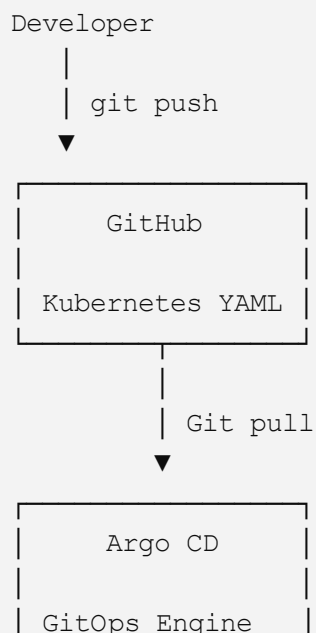
Kubernetes GitOps Lab — Complete Reference Guide

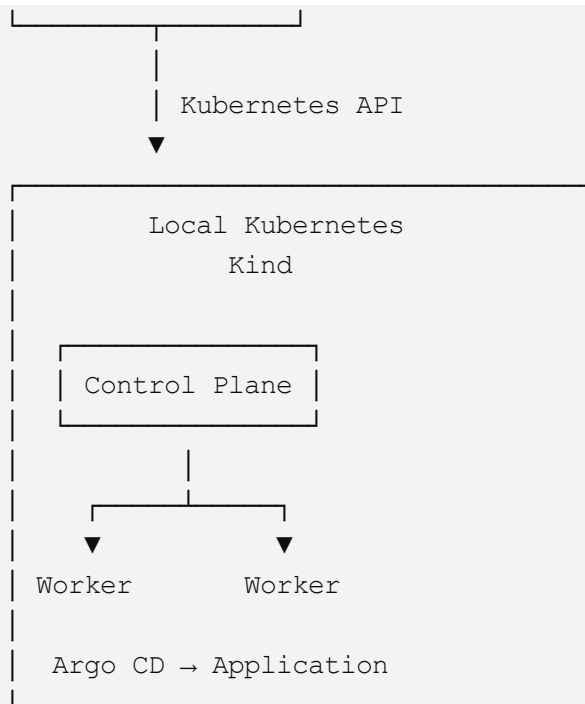
1. Lab Overview

This lab builds a complete local Kubernetes GitOps environment on a Windows laptop using:

- **Windows 11**
- **WSL2**
- **Ubuntu**
- **Docker Desktop**
- **Docker Engine**
- **Kind (Kubernetes IN Docker)**
- **Kubernetes**
- **Terraform**
- **kubectl**
- **Git / GitHub**
- **Argo CD**

The objective is to learn the workflow used in modern Kubernetes environments:





2. Overall Architecture

The complete environment looks like this:

Mermaid

flowchart TB

DEV["Developer / Laptop"]

subgraph WINDOWS["Windows Host"]

DD["Docker Desktop 4.86.0"]

subgraph WSL["WSL2"]

UB["Ubuntu"]

DOCKER["Docker Engine"]

TF["Terraform 1.15.8"]

KUBECTL["kubectl"]

KIND["Kind 0.32.0"]

GIT["Git"]

end

end

subgraph GITHUB["GitHub"]

REPO["k8s-gitops-lab
Git Repository"]

APP["app/
Application Manifests"]

ARGO["argocd/
Argo CD Application"]

end

subgraph KINDCLUSTER["Kind Kubernetes Cluster"]

CP["Control Plane
k8s-lab-control-plane"]

```

W1["Worker<br/>k8s-lab-worker"]
W2["Worker<br/>k8s-lab-worker2"]

subgraph ARGOCD["argocd namespace"]
    SERVER["argocd-server"]
    REPO["argocd-repo-server"]
    CTRL["application-controller"]
    REDIS["redis"]
    DEX["dex-server"]
end

subgraph DEFAULT["default namespace"]
    WEB["Web Application"]
end

end

DEV --> UB
DD --> DOCKER
UB --> DOCKER

TF --> KIND
KIND --> CP
KIND --> W1
KIND --> W2

GIT --> GITHUB
GITHUB --> SERVER
SERVER --> CTRL
CTRL --> WEB

CP --> ARGOCD
W1 --> ARGOCD
W2 --> ARGOCD

```

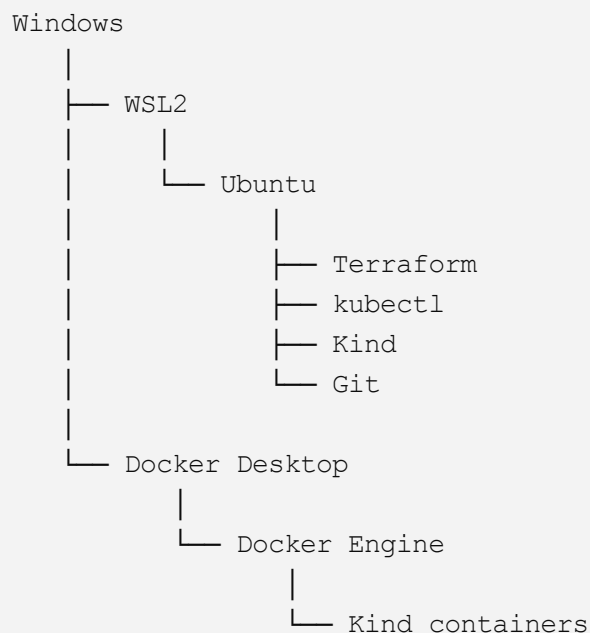
3. Technology Stack

Component	Version / Value	Purpose
Windows	Windows 11	Host OS
WSL	2.5.9.0	Linux environment
WSL Kernel	6.6.87.2	Linux kernel
Ubuntu	WSL2	Linux development environment
Docker Desktop	4.86.0	Container runtime

Component	Version / Value	Purpose
Docker Engine	29.7.2	Container execution
Terraform	1.15.8	Infrastructure provisioning
Kind	0.32.0	Local Kubernetes cluster
Kubernetes	v1.34.0	Container orchestration
kubectl	v1.36.3	Kubernetes CLI
Node image	kindest/node:v1.34.0	Kubernetes node image
GitHub	zhasaan/k8s-gitops-lab	Git repository
Argo CD	Installed in <code>argocd</code> namespace	GitOps deployment

4. Host Architecture

The lab uses:



The important point is that **Kind nodes are containers**, not virtual machines.

Your Kubernetes nodes are:

Docker Container

```
|
|— k8s-lab-control-plane
|— k8s-lab-worker
|— k8s-lab-worker2
```

5. WSL2 Verification

From PowerShell:

PowerShell

```
wsl --status
```

Expected:

```
Default Distribution: Ubuntu
Default Version: 2
```

Check distributions:

PowerShell

```
wsl -l -v
```

Current environment:

NAME	STATE	VERSION
Ubuntu	Running	2
docker-desktop	Running	2

Check WSL version:

PowerShell

```
wsl --version
```

Lab environment:

```
WSL version: 2.5.9.0
Kernel version: 6.6.87.2-1
WSLg version: 1.0.66
Windows version: 10.0.26200.9168
```

6. Docker Desktop

Docker Desktop provides the Docker Engine used by Kind.

The installed Docker Desktop version was verified using PowerShell:

PowerShell

```
(Get-Item "$env:LOCALAPPDATA\Programs\DockerDesktop\Docker Desktop.exe").Version  
Select-Object ProductVersion, FileVersion
```

Result:

```
ProductVersion FileVersion  
-----  
4.86.0.236216 4.86.0.236216
```

Docker Engine:

PowerShell

```
docker --version
```

Result:

```
Docker version 29.7.2
```

7. Docker Context

On Windows:

PowerShell

```
docker context ls
```

The active context was:

```
desktop-linux
```

This is important because Kind needs access to the Docker daemon.

From Ubuntu:

Bash

```
docker version
```

Expected:

```
Client:
  Version:           29.7.2
  ...

Server: Docker Desktop 4.86.0
Engine:
  Version:           29.7.2
  ...
```

The critical test is that **both Client and Server appear**.

If you only see:

```
Client:
...
failed to connect to the docker API
```

Docker Desktop's Linux engine is not available to Ubuntu.

8. Docker Desktop / WSL Troubleshooting

During the lab we encountered:

```
WSL integration with distro 'Ubuntu' unexpectedly stopped.
```

and:

```
install: cannot stat
'/mnt/wsl/docker-desktop/docker-desktop-user-distro':
No such file or directory
```

Initially:

```
Bash

docker ps
```

returned:

```
failed to connect to the docker API at unix:///var/run/docker.sock
```

After Docker Desktop was restarted and WSL integration recovered, the environment became:

```
Ubuntu          Running  2
docker-desktop  Running  2
```

and:

Bash

```
docker version
```

successfully showed both client and server.

Diagnostic commands

PowerShell:

PowerShell

```
wsl -l -v
wsl --status
wsl --version
docker version
docker info
docker context ls
```

Ubuntu:

Bash

```
docker version
docker info
docker ps
ls -l /var/run/docker.sock
mount | grep docker
```

9. Terraform Installation

Terraform was installed from the HashiCorp APT repository.

Version:

Bash


```
terraform --version
```

Result:

```
Terraform v1.15.8  
on linux_amd64
```

APT repositories included:

```
https://apt.releases.hashicorp.com
```

10. Terraform Directory Structure

The Terraform project is located at:

```
/home/ziazxx/terraform/k8-s
```

Directory:

```
terraform/  
├── k8-s/  
│   ├── versions.tf  
│   └── kind.tf
```

Check:

Bash

```
cd ~/terraform/k8-s  
ls -la
```

11. Terraform Provider Configuration

File:

```
versions.tf
```

Contents:

```
hcl  
  
terraform {
```

```
required_version = ">= 1.15.0"

required_providers {
  kind = {
    source  = "tehcyx/kind"
    version = "~> 0.11"
  }
}

provider "kind" {}
```

This tells Terraform to use the Kind provider:

```
tehcyx/kind
```

12. Kind Cluster Configuration

File:

```
kind.tf
```

Current configuration:

```
hcl

resource "kind_cluster" "lab" {
  name           = "k8s-lab"
  wait_for_ready = true
  node_image     = "kindest/node:v1.34.0"

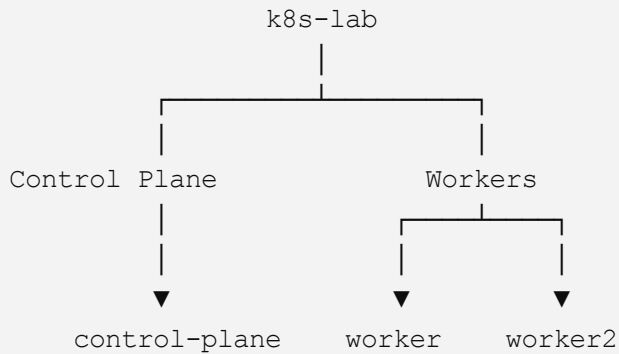
  kind_config {
    kind      = "Cluster"
    api_version = "kind.x-k8s.io/v1alpha4"

    node {
      role = "control-plane"
    }

    node {
      role = "worker"
    }

    node {
      role = "worker"
    }
  }
}
```

This creates:



13. Why `kindest/node:v1.34.0` ?

Kind does not create a Kubernetes node from a normal Ubuntu or Debian image.

The image:

```
kindest/node:v1.34.0
```

is a specially prepared container image containing the components required for a Kubernetes node.

You manually pulled it with:

Bash

```
docker pull kindest/node:v1.34.0
```

Docker showed:

```
kindest/node:v1.34.0
```

with approximately:

```
1.45GB disk usage  
445MB content size
```

The same image is used for:

```
control-plane  
worker  
worker2
```

14. Kind Version

Bash

```
kind version
```

Result:

```
kind v0.32.0 go1.26.3 linux/amd64
```

15. Terraform Initialization

From:

Bash

```
cd ~/terraform/k8-s
```

run:

Bash

```
terraform init
```

Terraform downloads the required provider.

Then:

Bash

```
terraform fmt  
terraform validate
```

Expected:

```
Success! The configuration is valid.
```

16. Terraform Plan

Run:

Bash

```
terraform plan
```

Expected result:

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

Terraform understands the desired topology:

```
1 control-plane  
2 workers
```

and:

```
node_image = kindest/node:v1.34.0
```

17. Creating the Kubernetes Cluster

Apply the configuration:

Bash

```
terraform apply
```

Confirm with:

```
yes
```

Terraform then creates the Kind cluster.

Verify:

Bash

```
kind get clusters
```

Expected:

```
k8s-lab
```

18. Docker View of Kubernetes

Because Kind runs Kubernetes nodes as Docker containers:

Bash

```
docker ps
```

showed:

```
k8s-lab-control-plane  
k8s-lab-worker  
k8s-lab-worker2
```

The control-plane container exposed Kubernetes API port:

```
127.0.0.1:<port> -> 6443
```

This explains why:

Bash

```
kubectl cluster-info
```

reported something similar to:

```
Kubernetes control plane is running at  
https://127.0.0.1:37425
```

19. Kubernetes Cluster

Check the current context:

Bash

```
kubectl config current-context
```

Result:

```
kind-k8s-lab
```

Check nodes:

Bash

```
kubectl get nodes -o wide
```

Current cluster:

NAME	STATUS	ROLES	VERSION
k8s-lab-control-plane	Ready	control-plane	v1.34.0
k8s-lab-worker	Ready	<none>	v1.34.0
k8s-lab-worker2	Ready	<none>	v1.34.0

20. Kubernetes System Components

Check:

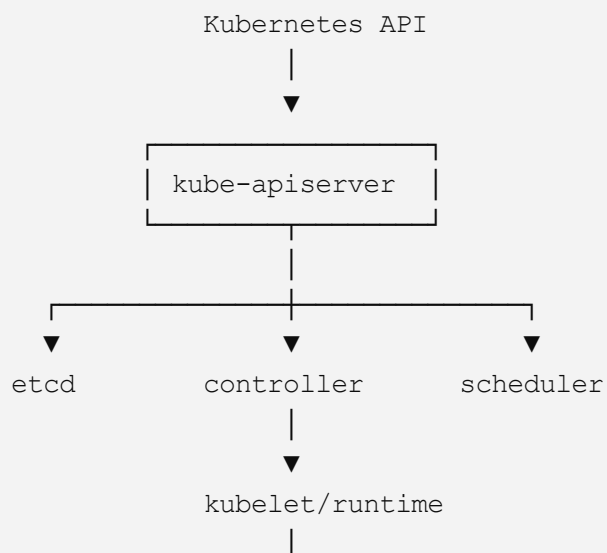
Bash

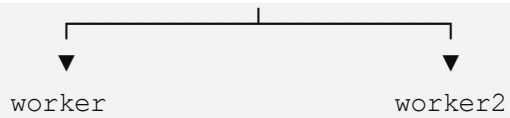
```
kubectl get pods -A
```

The cluster contains components including:

```
kube-apiserver
kube-controller-manager
kube-scheduler
etcd
coredns
kube-proxy
kindnet
local-path-provisioner
```

Architecture:





21. Kubernetes Namespaces

Bash

```
kubectl get ns
```

Initial namespaces included:

```
default
kube-node-lease
kube-public
kube-system
local-path-storage
```

Later we created:

```
argocd
```

for Argo CD.

22. Kubernetes CLI

The Kubernetes client was:

```
kubectl v1.36.3
```

Check:

Bash

```
kubectl version --client
```

Useful commands:

Bash

```
kubectl get nodes
kubectl get pods -A
kubectl get ns
```



```
kubectl cluster-info
kubectl config current-context
kubectl config get-contexts
```

Note:

This is incorrect:

Bash

```
kubectl get context
```

because a context is not a Kubernetes API resource.

Use:

Bash

```
kubectl config current-context
```

or:

Bash

```
kubectl config get-contexts
```

23. Installing Argo CD

Argo CD was installed into:

```
argocd
```

namespace.

Verify:

Bash

```
kubectl get pods -n argocd
```

The lab contains:

```
argocd-application-controller
argocd-applicationset-controller
argocd-dex-server
argocd-notifications-controller
```

```
argocd-redis
argocd-repo-server
argocd-server
```

All were successfully running.

24. Argo CD Architecture

Mermaid

flowchart LR

```
GIT["GitHub<br/>k8s-gitops-lab"]

subgraph K8S["Kind Kubernetes Cluster"]
    SERVER["Argo CD Server"]
    REPO["Repo Server"]
    CTRL["Application Controller"]
    REDIS["Redis"]

    SERVER --> REPO
    REPO --> CTRL
    CTRL --> REDIS

    APP["Web Application"]
    CTRL --> APP
end

GIT --> REPO
SERVER --> CTRL
```

The key component is the:

```
argocd-application-controller
```

It compares:

```
Desired State
  ↓
Git repository

versus

Actual State
  ↓
Kubernetes cluster
```

25. Git Repository

GitHub repository:

```
zhasaan/k8s-gitops-lab
```

Local directory:

```
~/git/k8s-gitops-lab
```

Clone:

Bash

```
cd ~/git
```

```
git clone https://github.com/zhasaan/k8s-gitops-lab.git
```

Then:

Bash

```
cd k8s-gitops-lab
```

Verify:

Bash

```
git remote -v
```

Expected:

```
origin https://github.com/zhasaan/k8s-gitops-lab.git
```

26. Git Repository Structure

The repository is evolving into:

```
k8s-gitops-lab/
|
├─ app/
|   ├── deployment.yaml
|   ├── service.yaml
|   └─ ...
|
```

```
|— argocd/
|   |— web-app.yaml
|
|— docs/
|   |— KUBERNETES-GITOPS-LAB.md
```

The important separation is:

```
app/
  Kubernetes application manifests

argocd/
  Argo CD configuration

docs/
  Documentation
```

27. Git Workflow

The normal development workflow is:

```
Edit YAML
  |
  ▼
git status
  |
  ▼
git add
  |
  ▼
git commit
  |
  ▼
git push
  |
  ▼
GitHub
  |
  ▼
Argo CD detects change
  |
  ▼
Kubernetes updated
```

Example:

Bash

```
git status
git add .
git commit -m "Update web application"
git push origin main
```

28. Argo CD Application

The Argo CD application definition is:

```
argocd/web-app.yaml
```

The important configuration is:

YAML

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: web-app
  namespace: argocd
spec:
  project: default

  source:
    repoURL: https://github.com/zhasaan/k8s-gitops-lab.git
    targetRevision: main
    path: app

  destination:
    server: https://kubernetes.default.svc
    namespace: default

  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

This tells Argo CD:

Get the Kubernetes manifests from the `app` directory of the `main` branch of GitHub and deploy them into the `default` Kubernetes namespace.

29. GitOps Flow

The resulting architecture is:

Mermaid

sequenceDiagram

```
participant DEV as Developer
participant GIT as GitHub
participant ARGO as Argo CD
participant K8S as Kubernetes
participant APP as Web App

DEV->>GIT: git push
GIT-->>ARGO: Repository state changes
ARGO->>GIT: Fetch desired state
ARGO->>K8S: Compare desired vs actual
K8S-->>ARGO: Current state
ARGO->>K8S: Apply manifests
K8S->>APP: Create/update resources
APP-->>K8S: Running
```

This is the core GitOps model.

30. First Argo CD Application

Initially:

Bash

```
kubectl get applications -n argocd
```

returned:

```
No resources found in argocd namespace.
```

After creating the Argo CD application:

Bash

```
kubectl apply -f argocd/web-app.yaml
```

the application appeared:

NAME	SYNC STATUS	HEALTH STATUS
------	-------------	---------------

31. Important Argo CD Authentication Issue

The first attempt resulted in:

```
ComparisonError
```

with:

```
failed to load target state:
failed to generate manifest

failed to list refs:
authentication required:
Repository not found.
```

This is an important distinction.

Your local Git client could access GitHub because you authenticated Git locally.

Argo CD is a **separate client**.

Therefore:

```
Your Ubuntu Git credentials
      ≠
Argo CD Git credentials
```

Argo CD needs its own repository credentials when the repository is private.

32. Argo CD Repository Authentication Model

The architecture is:

```
Git CLI
  |
  | credentials
  ▼
GitHub

Argo CD
  |
  | separate credentials
```

▼
GitHub

If the repository is private, Argo CD needs permission to read it.

The error:

```
authentication required
```

is therefore expected when no repository credentials have been configured.

33. Recommended GitHub Authentication

For a lab, use a GitHub Personal Access Token rather than a GitHub account password.

Do not put the token directly into:

```
web-app.yaml
```

or any Git-tracked file.

Instead, configure the credential as an Argo CD repository secret.

Conceptually:

```
GitHub
  ▲
  | HTTPS + token
  |
Argo CD repo-server
  ▲
  |
Kubernetes Secret
```

34. Current Argo CD Status

At the point reached in the lab:

Bash

```
kubectl get applications -n argocd
```

returned:

NAME	SYNC STATUS	HEALTH STATUS
web-app	Unknown	Healthy

This means:

```
Health = Healthy
```

does **not** mean the Git synchronization is working.

The important state is:

```
SYNC STATUS = Unknown
```

because Argo CD could not authenticate against the repository.

After repository authentication is configured, we should expect:

SYNC STATUS	HEALTH STATUS
Synced	Healthy

35. Verify Argo CD Application

Useful commands:

Bash

```
kubectl get applications -n argocd
```

Detailed:

Bash

```
kubectl describe application web-app -n argocd
```

Application YAML:

Bash

```
kubectl get application web-app -n argocd -o yaml
```

Watch:

Bash

```
kubectl get application web-app -n argocd -w
```

36. Verify the Application

Once Argo CD successfully syncs:

Bash

```
kubectl get all -n default
```

Also:

Bash

```
kubectl get deployments  
kubectl get pods  
kubectl get services
```

For example:

Bash

```
kubectl get pods -o wide
```

37. Terraform vs Argo CD

This is one of the most important architectural concepts in this lab.

Terraform

Terraform should primarily manage:

Infrastructure

For this lab:

```
Kind cluster  
Control plane  
Workers  
Cluster infrastructure
```

Argo CD

Argo CD should primarily manage:

Applications

For example:

Deployment
Service
ConfigMap
Ingress
Application configuration

Therefore:

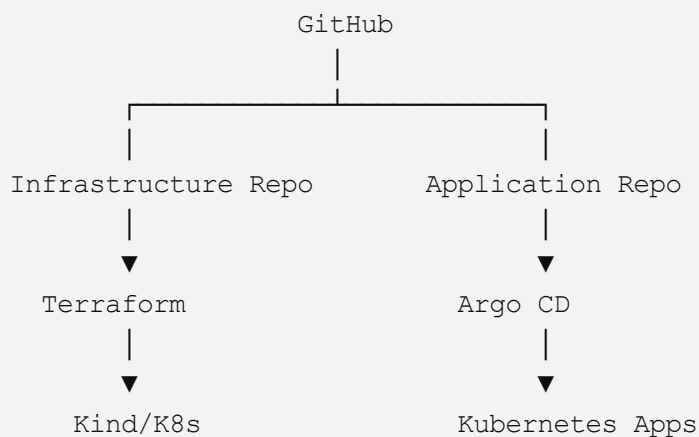
Terraform
|
└─ Creates Kubernetes infrastructure

Argo CD
|
└─ Deploys applications into Kubernetes

Avoid using Terraform to manage every application YAML file if the goal is to learn GitOps.

38. Recommended Future Architecture

The lab should eventually look like:



For a learning environment, however, keeping everything in one repository is perfectly reasonable:

k8s-gitops-lab/
|

```
| terraform/  
|  
| app/  
|  
| argocd/  
|  
| docs/
```

39. Recommended Lab Repository

A good final repository structure would be:

```
k8s-gitops-lab/  
|  
| README.md  
|  
| docs/  
|   KUBERNETES-GITOPS-LAB.md  
|  
| terraform/  
|   kind/  
|     versions.tf  
|     kind.tf  
|     variables.tf  
|     outputs.tf  
|  
| argocd/  
|   web-app.yaml  
|   project.yaml  
|  
| app/  
|   namespace.yaml  
|   deployment.yaml  
|   service.yaml  
|   ingress.yaml
```

This will make the repository much easier to expand.

40. End-to-End Lab Lifecycle

Step 1 — Start Docker Desktop

Verify:

```
PowerShell
```

```
docker info
```

Step 2 — Start WSL

PowerShell

```
wsl -d Ubuntu
```

Step 3 — Verify Docker

Bash

```
docker version  
docker ps
```

Step 4 — Verify Kind

Bash

```
kind version  
kind get clusters
```

Step 5 — Verify Terraform

Bash

```
terraform --version
```

Step 6 — Verify Kubernetes

Bash

```
kubectl config current-context  
kubectl get nodes
```

Expected:

```
kind-k8s-lab
```

Step 7 — Verify Argo CD

Bash

```
kubectl get pods -n argocd
```

All Argo CD components should be:

```
Running
```

Step 8 — Verify Git

Bash

```
cd ~/git/k8s-gitops-lab
git status
git remote -v
```

Step 9 — Modify Application

Edit:

```
app/
```

Step 10 — Commit

Bash

```
git add .
git commit -m "Update web application"
git push origin main
```

Step 11 — Argo CD Synchronizes

Argo CD detects:

```
Git desired state changed
```

and applies the changes to Kubernetes.

41. Operational Verification Checklist

Use this checklist whenever you start the lab.

Host

Bash

```
wsl --status  
wsl -l -v
```

Docker

Bash

```
docker version  
docker info  
docker ps
```

Kind

Bash

```
kind version  
kind get clusters
```

Kubernetes

Bash

```
kubectl config current-context  
kubectl get nodes  
kubectl get pods -A
```

Argo CD

Bash

```
kubectl get pods -n argocd
kubectl get applications -n argocd
```

Git

Bash

```
cd ~/git/k8s-gitops-lab
git status
git log --oneline
git remote -v
```

42. Troubleshooting Matrix

Problem	Check	Likely Cause
<code>docker ps</code> cannot connect	<code>docker info</code>	Docker Desktop not running
<code>/var/run/docker.sock</code> missing	<code>ls -l /var/run/docker.sock</code>	WSL Docker integration unavailable
Kind cannot create cluster	<code>docker ps</code>	Docker Engine unavailable
Nodes not Ready	<code>kubectl get nodes</code>	Kubernetes startup/network issue
Argo CD pods not Running	<code>kubectl get pods -n argocd</code>	Argo CD installation issue
Application <code>Unknown</code>	<code>kubectl describe application</code>	Repo/API/configuration issue
<code>authentication required</code>	Argo CD Application status	GitHub credentials missing
<code>Repository not found</code>	Verify repo URL	Private repo or incorrect URL
Git push asks for password	Git remote	HTTPS authentication
<code>kubectl get context</code> fails	Use <code>kubectl config</code>	Context isn't a Kubernetes resource

43. Important Kubernetes Commands

Nodes

Bash

```
kubectl get nodes  
kubectl get nodes -o wide
```

Pods

Bash

```
kubectl get pods  
kubectl get pods -A  
kubectl get pods -o wide
```

Services

Bash

```
kubectl get svc  
kubectl get svc -A
```

Deployments

Bash

```
kubectl get deployments  
kubectl get deployments -A
```

Namespaces

Bash

```
kubectl get ns
```

Everything

Bash

```
kubectl get all
```

Describe

Bash

```
kubectl describe pod <pod>
kubectl describe deployment <deployment>
```

Logs

Bash

```
kubectl logs <pod>
```

44. Important Terraform Commands

Bash

```
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
terraform destroy
```

Typical lifecycle:

```
terraform init
    ↓
terraform fmt
    ↓
terraform validate
    ↓
terraform plan
    ↓
terraform apply
    ↓
Kind Kubernetes cluster
```

Destroy the lab:

Bash

```
terraform destroy
```

This should remove the Kind cluster managed by Terraform.

45. Important Kind Commands

Bash

```
kind version
kind get clusters
kind create cluster
kind delete cluster
```

Check the underlying containers:

Bash

```
docker ps
```

The Kind cluster:

```
k8s-lab
```

corresponds to:

```
k8s-lab-control-plane
k8s-lab-worker
k8s-lab-worker2
```

46. Desired Final GitOps Architecture

The long-term learning architecture should be:

Mermaid

```
flowchart TB
```

```
DEV["Developer"]
```

```
subgraph SCM["GitHub"]
```

```
    GIT["k8s-gitops-lab"]
```

```
    MANIFEST["Kubernetes Manifests"]
```

```
    ARGOFILE["Argo CD Application"]
```

```
end
```

```
subgraph INFRA["Terraform"]
```

```
    TF["Terraform"]
```

```
    KIND["Kind"]
```

```
end
```

```
subgraph CLUSTER["Kubernetes / Kind"]
```

```
    CP["Control Plane"]
```

```
W1["Worker"]
W2["Worker"]

subgraph ARGO["Argo CD"]
  AS["Argo CD Server"]
  RS["Repo Server"]
  AC["Application Controller"]
end

subgraph APPLICATION["Application"]
  DEP["Deployment"]
  SVC["Service"]
  POD["Application Pods"]
end

DEV -->|"git commit / push"| GIT
GIT --> MANIFEST
GIT --> ARGOFILE
```

47. What You Have Learned So Far

You have already covered a significant amount of real-world infrastructure:

Infrastructure

- WSL2
- Docker Desktop
- Docker Engine
- Docker contexts
- Kind
- Kubernetes nodes
- Kubernetes networking

Infrastructure as Code

- Terraform
- Terraform providers
- Terraform resources

- `terraform init`
- `terraform validate`
- `terraform plan`
- `terraform apply`

Kubernetes

- Control plane
- Workers
- Pods
- Services
- Deployments
- Namespaces
- kubeconfig
- contexts
- CoreDNS
- kube-proxy
- containerd

Git

- GitHub repository
- Git clone
- Git remote
- Git branch
- Git commits
- Git push
- Repository structure

GitOps

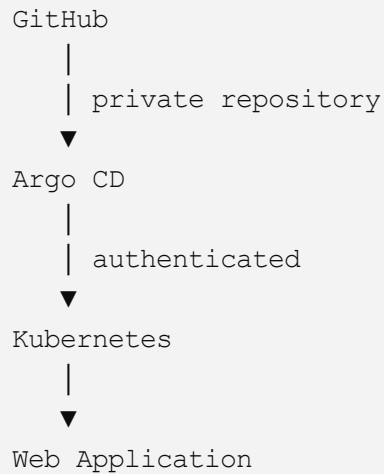
- Argo CD
- Argo CD Application
- Git as desired state
- Automated sync
- Self-healing
- Pruning
- Repository authentication

That is already a very good foundation.

48. Next Stage of the Lab

I recommend we **do not add more tooling yet**.

The next milestone should be:



Specifically, we should finish the current problem:

```
web-app
SYNC STATUS: Unknown
HEALTH STATUS: Healthy
```

by configuring **Argo CD's GitHub repository authentication**.

Then we should verify:

Bash

```
kubectl get applications -n argocd
```

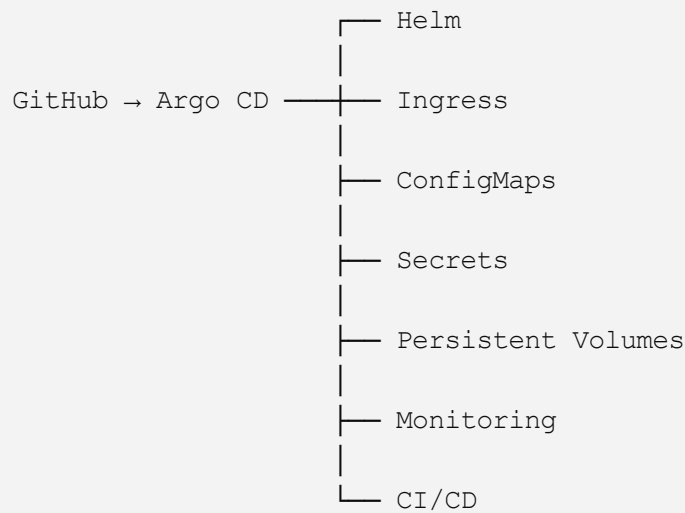
becomes:

NAME	SYNC STATUS	HEALTH STATUS
web-app	Synced	Healthy

After that, the really useful GitOps exercise is:

1. Change replica count in Git
2. git commit
3. git push
4. Argo CD detects change
5. Kubernetes changes automatically
6. Verify with kubectl

Then we can move on to:



That will turn the current cluster into a genuinely useful **DevOps/Kubernetes/GitOps training platform** rather than just a Kubernetes installation.

Suggested title for the repository document

Kubernetes GitOps Lab – Terraform, Kind, Kubernetes, GitHub and Argo CD

And I would keep this document under:

```
~/git/k8s-gitops-lab/docs/KUBERNETES-GITOPS-LAB.md
```

Then commit it:

Bash

```
cd ~/git/k8s-gitops-lab

mkdir -p docs

# save the document as:
# docs/KUBERNETES-GITOPS-LAB.md

git add docs/KUBERNETES-GITOPS-LAB.md
git commit -m "Add comprehensive Kubernetes GitOps lab documentation"
git push origin main
```

One important correction before you proceed: don't put your GitHub username/password/token into this document or into Git. Your Argo CD repository credential should be stored as a Kubernetes/Argo CD secret, and the repository authentication problem is the next thing we should resolve.

